

10. Technical Architecture Specification

Target architecture for the private medium-term portfolio decision platform

Project: QuantPipeline

Document status: Working baseline for product and engineering delivery

Version: v1.0

Date: 2026-04-23

Audience: Product owner, architecture, development, QA, and future implementation agents

Purpose: This document is intended to be implementable. It defines contracts, boundaries, and decisions that should survive future feature growth without breaking the core recommendation pipeline.

Document Control

Field	Value
Document code	10
Title	Technical Architecture Specification
Primary input set	Documents 01-09, A1 technology recommendation, and baseline truth table
Primary architecture anchor	Data & Feature Layer -> Signals -> Selection -> Allocation -> Timing -> Risk Overlay -> Recommendation Object -> Backtest / Paper / Replay / Ops
Assumed release posture	Private single-user first release with future-safe modular boundaries
Key dependencies	A1 Technology Recommendation, 08 PRD, 09 Functional Requirements
Quality rule	No silent degradation, explicit trust semantics, and replayable output

1. Purpose and Architectural Scope

This specification translates the product direction into a buildable architecture. It defines the major runtime layers, service boundaries, data and control flows, deployment shape, and cross-cutting non-functional rules required to implement QuantPipeline as a private portfolio decision platform for medium-term investing.

The architecture is intentionally recommendation-centric. The platform is not organized around disconnected research widgets. It is organized around one canonical pipeline that converts heterogeneous evidence into a unified recommendation object, preserves the stages that produced it, and exposes that evidence consistently across user experience, replay, backtests, paper shadowing, and admin operations.

The scope of this document covers logical architecture, runtime flows, deployment topology, service responsibilities, resilience, observability, security, and change control. Detailed schemas and endpoint definitions are delegated to Documents 11 and 12, but the architecture here governs their boundaries and interactions.

2. Architecture Principles

- One canonical recommendation contract across production, replay, backtests, and paper.
- Clear separation between evidence generation and portfolio policy decisions.
- Explicit handling of degraded, stale, and suppressed conditions rather than silent fallback.
- Service modularity with low-coupling contracts and stable domain identifiers.
- Replayability and auditability by design, not as a later add-on.
- Desktop-primary experience with credible mobile access and consistent API behavior.
- Batch-oriented analytical pipelines combined with low-latency read APIs for the user interface.
- Operational honesty: system health must be reflected in what the user sees.

3. Assumed First-Release Context

The first release is private and single-user in product posture, but the architecture should not hard-code single-user shortcuts that would make future administrative separation or role-based access impossible. This means explicit authentication, explicit audit trails, and explicit ownership of scenarios, recommendations, approvals, and overrides.

The architecture assumes moderate scale in user count but meaningful scale in data processing breadth: daily and intraday bars, news and text events, sentiment updates, feature recalculation, multiple policy variants, backtest jobs, and continuous paper monitoring. The limiting factor is not concurrent public traffic; it is analytical correctness, operational clarity, and reproducibility of decisions.

4. Target Logical Architecture

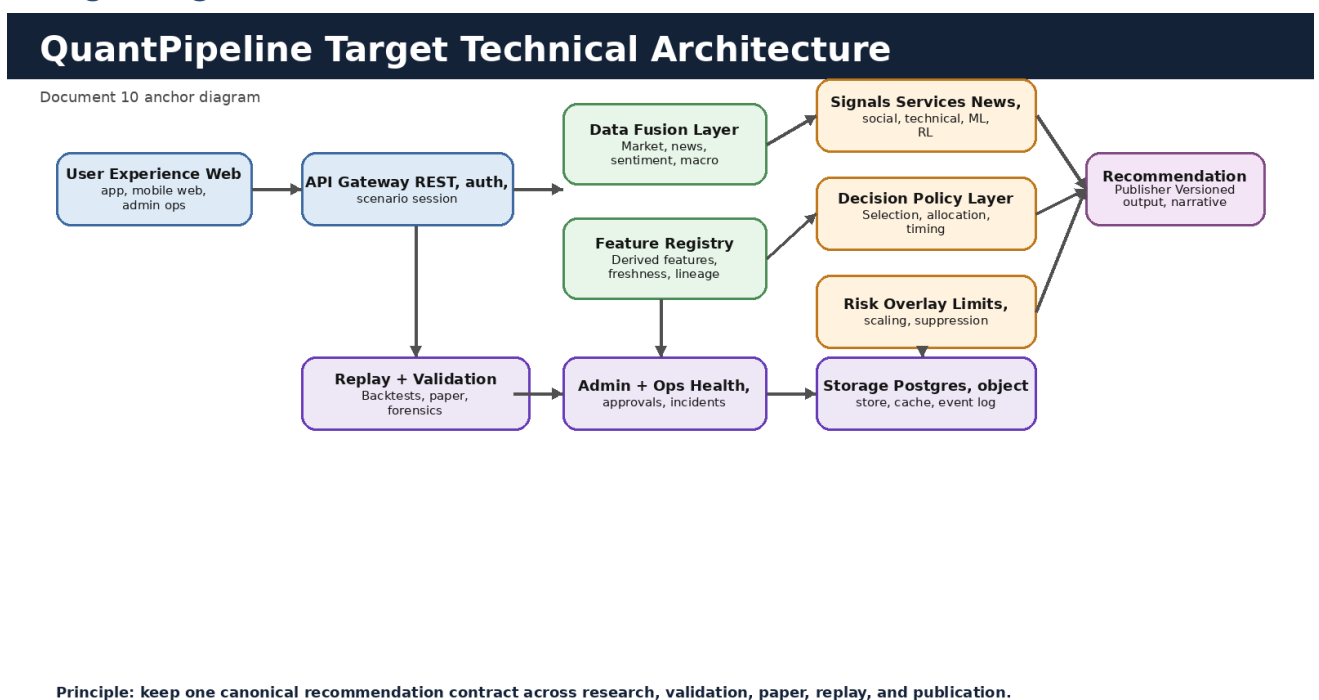


Figure 10-1. Recommended target architecture.

Target Architecture

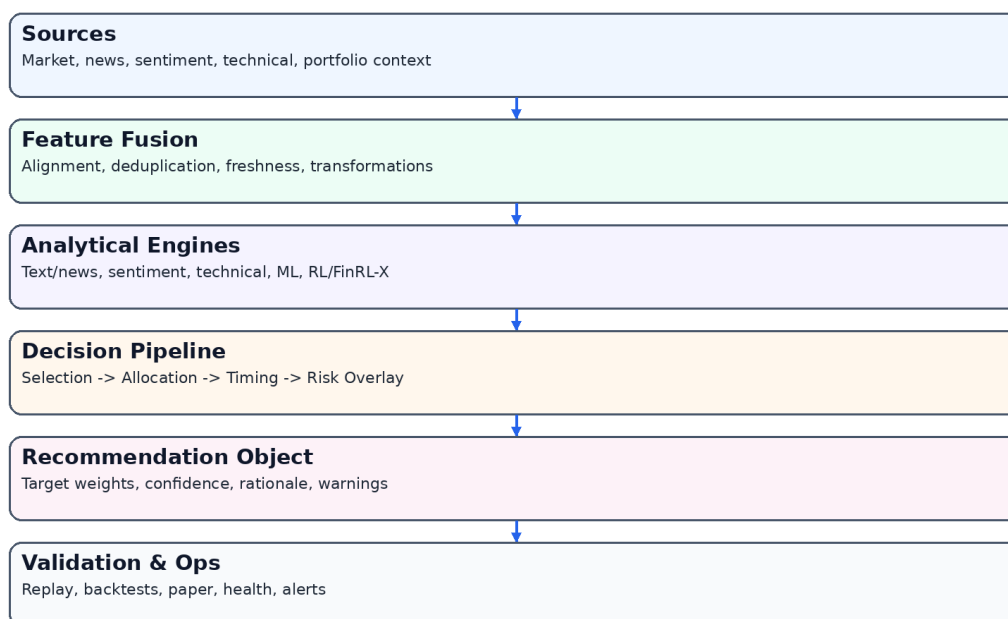


Figure 10-2. Prior architecture concept retained as continuity reference.

The platform should be read from left to right. The user experience layer consumes read-optimized APIs; the APIs orchestrate access to recommendation state, decision-stage evidence, and validation outputs; the analytical core constructs policy outputs in a deterministic order; and the storage layer preserves enough detail to recreate both current state and historical runs.

5. Major Architectural Layers

Each architectural layer owns a distinct responsibility and should have a clear contract. Implementation may consolidate layers into fewer deployable services at the beginning, but the conceptual boundaries must remain intact so the platform can grow without redesigning the decision lifecycle.

Layer	Primary responsibility	Input examples	Output examples
Experience layer	Present current recommendation, breakdown, validation, and ops state	Scenario filters, selected recommendation id	Rendered overview, dashboard data, admin actions
API gateway	Authenticate, authorize, compose read models, and issue write commands	HTTP requests, session tokens	Versioned JSON responses, accepted jobs
Data fusion	Normalize raw	Vendor payloads,	Curated time-aligned

Layer	Primary responsibility	Input examples	Output examples
	market, text, and macro feeds	scraped text, batch imports	inputs
Feature registry	Persist derived features with freshness and lineage	Curated inputs, feature jobs	Feature sets, completeness markers, provenance
Signal services	Generate engine-level outputs from news, social, technical, ML, RL	Feature sets, model artifacts	Signal runs, confidence, rationale fragments
Decision policy layer	Convert signals into selection, allocation, timing, and pre-risk proposals	Signal runs, universes, policy settings	Decision-stage objects
Risk overlay	Apply portfolio constraints, stress rules, and suppressions	Pre-risk proposal, portfolio state, risk rules	Post-risk proposal, modifications, warnings
Recommendation publisher	Materialize current recommendation and publication state	Post-risk proposal, ops state	Published recommendation object, narrative, status
Validation and replay	Run historical evaluation and reconstruct past decisions	Frozen snapshots, policy variants	Backtests, paper records, replay narratives
Admin and ops	Track health, incidents, approvals, and change state	Alerts, jobs, approval actions	Ops dashboards, audit events, incident records

6. Service Decomposition Recommendation

For the first release, deployable simplicity matters. The recommended implementation is not one microservice per conceptual layer. Instead, use a small number of deployables with strong internal domain modules. This preserves architectural clarity without incurring orchestration overhead too early.

A pragmatic initial decomposition is: (1) Web experience service, (2) API and orchestration service, (3) Data and analytics worker service, (4) Validation worker service, and (5) Shared storage and cache infrastructure. These deployables may share a single repository while still respecting the domain modules described below.

Deployable	Contains	Why this grouping works
Web experience	React/Next.js app, authenticated pages, client-side state, charts	Keeps the user experience release cycle independent from heavy analytical jobs
API/orchestration	REST API, session management,	Owns user-facing contracts and policy publication

Deployable	Contains	Why this grouping works
	recommendation read models, command handlers	workflow
Analytics worker	Ingestion, feature jobs, signal generation, selection/allocation/timing/risk jobs	Allows batch scheduling and compute isolation
Validation worker	Backtests, ablations, replay precomputation, paper portfolio maintenance	Separates longer-running evaluation work from operational recommendation jobs
Shared platform	PostgreSQL, object storage, Redis/cache, scheduler, logs, metrics	Centralizes persistence, coordination, and operational evidence

7. Runtime Modes

The architecture must support four distinct runtime modes that share contracts but differ in latency expectations, data windows, and publication behavior.

Mode	Trigger	Latency expectation	Publishes user-visible output?
Production recommendation run	Scheduled or operator-triggered	Minutes, deterministic	Yes, if policy and ops gates pass
Scenario simulation	User-initiated	Seconds to low minutes	No, returns temporary scenario output
Replay reconstruction	User-initiated from history	Seconds	No, read-only reconstruction
Backtest / validation	Operator or release workflow	Minutes to hours	No direct publication; results feed validation surfaces
Paper shadow update	Scheduled with market and policy cadence	Minutes	Yes, to paper portfolio surfaces only

8. Canonical Production Flow

1. Acquire or refresh raw inputs from market, text, sentiment, macro, and portfolio sources.
2. Normalize timestamps, enrich metadata, and store curated input batches with source lineage.
3. Compute or refresh feature sets. Mark completeness and freshness explicitly.
4. Run engine-level signal services. Persist confidence, rationale snippets, and warnings.
5. Execute selection policy on a defined universe.
6. Execute allocation policy on the shortlisted set.
7. Execute timing policy to adjust aggressiveness, delay, or exposure posture.
8. Execute risk overlay to enforce portfolio-level controls and suppression rules.
9. Publish a versioned recommendation object only if operational and policy gates pass.

10. Emit audit, metrics, and user-facing freshness or degraded state metadata.

Architectural rule: Selection, allocation, timing, and risk overlay must be stored as separate stage outputs. The final recommendation must never be the only preserved artifact.

9. Storage and Compute Architecture

QuantPipeline requires a hybrid persistence model. Relational storage should own canonical operational state, domain entities, decision-stage records, and read models. Object storage should own large immutable artifacts such as raw source payloads, model bundles, exported reports, and frozen backtest packages. Cache should be used only for performance and coordination, not as the system of record.

The compute model should prefer scheduled jobs with explicit run identifiers and bounded retries. Long-running backtests and paper analysis must never block recommendation publication or UI read performance.

Technology role	Recommended store	Notes
Canonical metadata and domain entities	PostgreSQL	Relational integrity, migrations, index control, and reporting flexibility
Raw payload archives and large artifacts	Object storage	Immutable storage keyed by source and snapshot ids
Short-lived cache and job coordination	Redis or equivalent	Used for deduplication, locks, and view caching
Observability events and metrics	Metrics store + log pipeline	Separate operational telemetry from product domain data
Search-oriented retrieval for text drill-down	Optional document index	Only after core contracts are stable

10. Deployment Topology

A multi-environment topology is required even for a private product. At minimum, maintain local development, integration/staging, and production. Validation environments can share infrastructure with staging but should preserve isolated datasets and separate experiment namespaces.

Environment	Purpose	Allowed data	Key controls
Local development	Feature development and manual debugging	Synthetic or scrubbed data	Developer-owned configs, seeded fixtures
Integration / staging	End-to-end verification and release rehearsal	Staged or delayed production-like data	Shared auth, smoke tests, migration rehearsal
Production	Live private usage and paper monitoring	Current operational data	Strict secrets management, audit, backups, alerting

Environment	Purpose	Allowed data	Key controls
Validation sandbox	Heavy backtest and policy experimentation	Frozen historical snapshots	No direct publication rights to production

11. Security Architecture

- All provider credentials, ingestion secrets, and model keys remain server-side only.
- Use authenticated sessions even for a private single-user product so that audit, admin separation, and remote access can remain disciplined.
- Store recommendation history, settings, and portfolio state as sensitive data with strict access logging.
- Separate read permissions from write permissions for publication, suppression, and policy changes.
- Require immutable audit events for recommendation publication, approval, suppression, and manual overrides.
- Apply signed or verified artifact references for model bundles and policy packages promoted to production.
- Protect replay and validation data from accidental mutation by using frozen snapshots and read-only references.

12. Observability and Operational Architecture

Observability must cover both machine health and decision health. Infrastructure metrics alone are insufficient because the product can appear technically alive while the recommendation pipeline is stale, suppressed, or semantically degraded.

The architecture should therefore produce structured metrics and logs for source freshness, feature completeness, signal coverage, policy execution, publication gates, API latency, cache hit rate, background job backlog, incident state, and user-visible suppression conditions.

Telemetry family	Examples	Consumer
Infrastructure	CPU, memory, disk, queue lag, API latency	Engineering and ops
Data health	Missing bars, missing news, stale macro data, vendor failures	Ops and admin
Decision health	Selection universe size, allocation feasibility, timing suppressions, risk overrides	Product owner and replay users
Publication health	Published / suppressed count, approval latency, freshness age	User-facing overview and admin
Validation health	Backtest queue, failed experiments, paper drift alerts	Operator and release governance

13. Resilience and Failure Strategy

The platform must fail in ways that preserve honesty. If one evidence family is unavailable, the system may continue only if policy rules explicitly allow degraded operation and the published output reflects that condition. If a stage is mandatory for publication, the pipeline must stop and mark the recommendation as suppressed or stale rather than silently substituting normal output.

Retries should be bounded and stage-specific. Repeated failures in ingestion or feature builds should not cascade into corrupted decision outputs. The recommended pattern is: isolate failing jobs, preserve prior healthy recommendation state, raise an admin-visible alert, and expose clear freshness and suppression signals to the user interface.

14. Performance and Scalability Expectations

Performance requirements are asymmetric. Read APIs for current recommendation, dashboard drill-down, and admin state should feel interactive. Production recommendation jobs may take minutes if they preserve determinism and evidence. Backtests may take longer as long as they are observable, cancellable, and isolated from operational flows.

Scalability should first optimize analytical concurrency and storage hygiene, not public web throughput. The main future risks are growing history, more engines, larger universes, richer replay artifacts, and validation load.

Capability	Initial expectation	Architecture implication
Overview load	Sub-2 second perceived response	Use read models and cache stable aggregates
Recommendation run	Low minutes, deterministic	Favor scheduled jobs with explicit orchestration
Scenario simulation	Interactive to low-minute	Reuse cached features and last healthy signal runs when valid
Backtest batch	Minutes to hours	Worker isolation, artifact storage, resumable jobs
Historical archive	Multi-year retention	Partition tables and externalize bulky artifacts

15. Architecture Decision Records and Change Control

Architecture changes that affect contracts, persistence boundaries, publication logic, or validation comparability should be captured in ADRs. This includes changing the recommendation object, adding a new evidence family, replacing an allocator, introducing a new suppression rule, or changing retention and snapshot semantics.

Every ADR should state the decision, motivation, alternatives considered, compatibility impact, migration plan, test impact, and rollback posture. This is especially important because the system is intended to evolve from a private product into a richer decision platform without losing continuity of historical evidence.

ADR category	Examples	Mandatory review?
Contract	Recommendation object, API envelope, id strategy	Yes
Policy	Selection rule set, timing controller, risk overlay semantics	Yes
Platform	Storage engine, queue model, deployment topology	Yes
Observability	New SLO, alert semantics, evidence retention	Usually
UX/ops coupling	Expose new degraded states or operator actions	Yes if user-visible

16. Non-Functional Traceability Matrix

The following matrix turns general architecture quality claims into implementation obligations.

Need	Architecture response	Primary owner
Replayability	Store stage outputs, snapshot ids, and immutable publication records	API + data domains
Operational honesty	Expose freshness, suppressions, and degraded states end-to-end	API + admin ops
Scalability	Partition storage and isolate heavy jobs	Platform
Security	Server-side secrets, RBAC, immutable audits	Platform + admin
Validation integrity	Frozen datasets and isolated experiment namespaces	Validation domain
Future mobile path	Stable APIs and scenario-safe payloads	Experience + API

Appendix A - Reference Sequence Flows

Reference flow 1: Scheduled recommendation run. Trigger scheduler -> ingest refresh -> feature build -> signal generation -> policy stages -> risk overlay -> publication gate -> publish read model -> audit event -> user-visible freshness update.

Reference flow 2: User scenario simulation. User changes horizon or allocator -> API validates scenario -> snapshot current curated inputs -> run selection/allocation/timing/risk in temporary mode -> return scenario result with clear non-published flag.

Reference flow 3: Replay request. User opens historical recommendation -> API loads publication record -> resolves linked stage outputs and evidence -> reconstructs narrative and diffs -> returns read-only response.

Reference flow 4: Backtest promotion. Operator starts experiment -> validation worker freezes inputs and config -> runs walk-forward tests -> stores metrics and artifacts -> governance rules evaluate pass/fail -> admin surface proposes or blocks policy promotion.

Appendix B - Open Architecture Decisions for Next Iteration

- Whether to introduce a dedicated feature store service or keep feature registry inside the analytics worker module.
- Whether text retrieval should use a dedicated search index in v1 or remain relational plus object-store based.
- Whether paper shadow rebalances should be computed by the validation worker or by a dedicated portfolio monitor service.
- Whether mobile push-style notifications are needed before a native app exists.
- Whether to split the API gateway from orchestration once scenario traffic and admin actions become materially different workloads.